ECR-01 — Replace the FS-i6X / FS-iA10B radio pair with a laptop-driven ESP32-S3 + DX-LR41 link

Project: Mojave Sphinx liquid bipropellant rocket — Build, Integration, and Launch Guidebook, HCR-5100

Affected section: Section VI — Ground Support Equipment (pages 235–283 of the PDF, document pages 236–284; §6.1 GSE Overview, §6.2 GSE Bill of Materials, §6.3 GSE Assembly Procedures, OP 1–OP 8)

Also affected: Section VII — Launch Preparation and Procedures (§7.2.2 ground system setup, §7.3.2 GSE startup and checkouts, §7.5.2 aborted firing)

Change class	Design change, control subsystem only. Fluid system (fill valve, QD, fill line, supply bottle) untouched.
Status	Proposed — for review before the next build/firing
Prepared	6 Nov 2026
Author	(to be filled in by the revising team)
Approval required from	GSE lead, launch operations (LCO/RSO interface), safety reviewer

1. Summary

The stock GSE control subsystem is a FlySky **FS-i6X** hobby transmitter (part `DTUS_FSI6X_10CH`) talking AFHDS 2A to an **FS-iA10B** receiver inside the toolbox. It works, it is cheap, and — importantly — it is not programmable, which is the reason for this change.

This ECR replaces that pair with:

- a **laptop** running a small Python GUI, connected by **USB-C** to
- an **ESP32-S3 "TX node"** in the GSE box, which drives
- a **DX-LR41-24T12S** 2.4 GHz LoRa module (Semtech SX1281, UART "transparent" pipe, AT-command configured);

over the 2.4 GHz link to

- a second **DX-LR41-24T12S** in the second **ESP32-S3 "pad node"**, which directly drives
- the three valve servos (FILL, VEHICLE Y-harness, PURGE) and the igniter relay.

Per the design decision taken with the requester, the FS-iA10B receiver is **not** retained: the pad node replaces it. The consequence — and the bulk of this document — is that every behaviour the receiver used to provide for free (PWM generation, channel mapping, spring-return semantics, and above all **failsafes**) must now be provided by the pad node and proven.

Channel numbering is kept faithful to the handbook and to the FS-iA10B harness so that an existing GSE box can be converted with no rewiring of the servo leads:

Function	Handbook channel (FS-iA10B)	Transmitter control	Pad-node output
FILL valve	Ch5	Switch B	D4

Function	Handbook channel (FS-iA10B)	Transmitter control	Pad-node output
VEHICLE valves (Y-harness)	Ch6	Switch D	D5
PURGE valve (optional)	Ch8	Switch A	D6
IGNITER relay	Ch3	Switch C + left-stick lockout	D7

The mapping is software-defined and re-programmable from the GUI at any time — the point of the exercise.

2. Reason for change

Requirement	Why the stock hardware cannot meet it
Programmable channel / switch mapping, endpoints and failsafe logic	FS-i6X exposes only what its firmware menus offer; endpoints are limited to ±120 %, failsafes are per-channel positions only, and there is no scripting, sequencing or conditional logic.
Scripted, repeatable command sequences (e.g. timed fill, automated abort, igniter pulse-width control)	No host interface. The i6X has a PS/2 trainer input and a firmware-update port, but no documented command API.
Computer logging of every command and every actuator state, time-stamped	Manual note-taking only.
Deterministic igniter on-time, one e-match pulse per press, hard limit	The PWM-controlled relay latches for as long as the channel is high; on-time is whatever the operator's finger does.
Long-range, low-cost, licence-free link that does not depend on hobby-RC stock	AFHDS 2A is a closed protocol. 2.4 GHz LoRa (SX1281) with a 2.4 GHz ISM band, no licence, and a UART "transparent" module needs no RF development.
Cheap, re-flashable, well-documented firmware	ESP32-S3 + Arduino/PlatformIO, ~$8 per node.

Non-goals: this change does **not** alter the fluid system, the valve hardware, the fill procedure, or the vehicle.

3. Scope

In scope (rewrite of the following handbook content):

- §6.1 GSE Overview — control-system description and Figure 6.1
- §6.2 GSE Bill of Materials — line items 1, 6, 9, 10 (and additions)
- §6.3 OP 6 Electrical System Integration — steps 1, 2, 7–15
- §6.3 OP 8 Transmitter Setup & Firmware Configuration — replaced in full by a two-part node bring-up procedure
- §7.2.2, §7.3.2, §7.5.2 — the operator-facing steps that change

Out of scope: fill system (OP 2–5), servo-actuated ball valve build (OP 1), cables and fill line (OP 7), the vehicle, and Sections I–V.

4. System description

4.1 Architecture

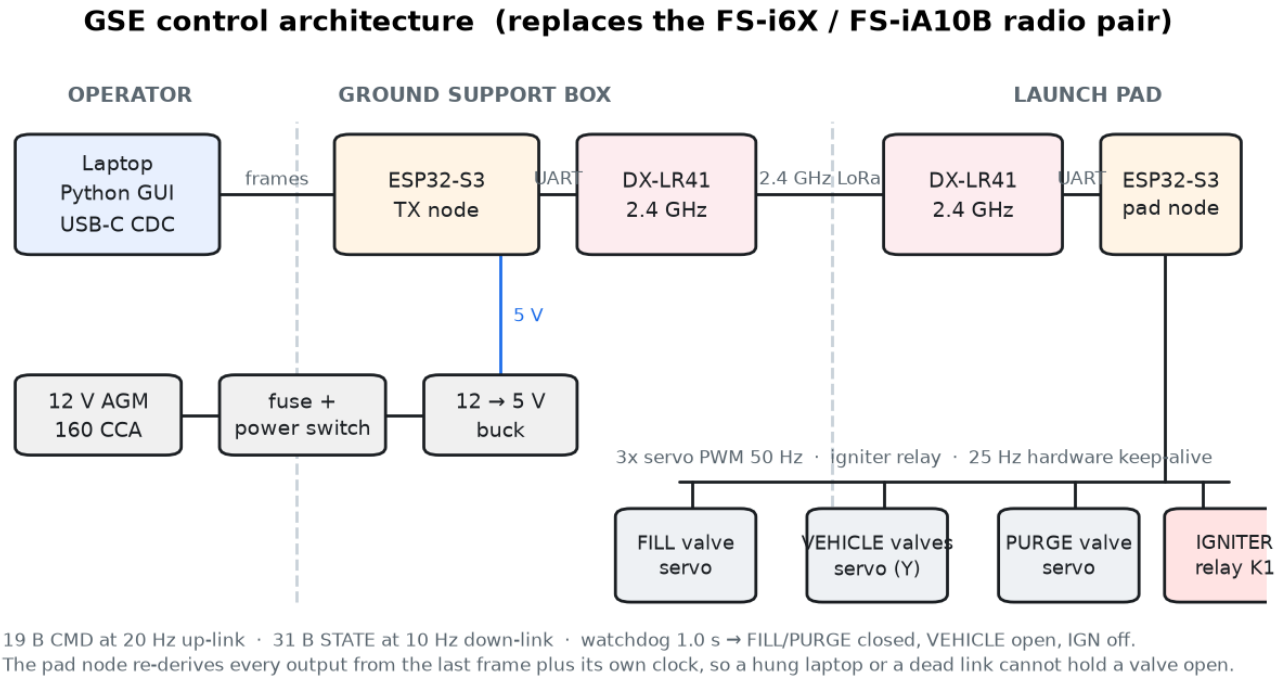


Figure 1 — GSE control architecture. The dashed lines mark the boundary between the operator (standoff distance) and the pad.

Two radio nodes, one protocol:

	TX node	Pad node
Hardware	ESP32-S3 devkit + DX-LR41-24T12S	ESP32-S3 devkit + DX-LR41-24T12S
Power	5 V from the laptop's USB-C port	5 V from the existing DROK 12→5 V regulator (12 V AGM)
Role	USB-CDC ↔ 2.4 GHz bridge, command pacing, stale-frame rejection	All safety logic, servo PWM, igniter sequencing, telemetry
Failure mode	Laptop dies → node transmits synthetic SAFE commands	Node dies → servos hold, igniter coil drops, outputs reboot safe

The two nodes are deliberately asymmetric: the pad node is the only device that makes decisions, and it makes them from its own clock, not from anything the laptop says.

4.2 Why the DX-LR41 cannot simply talk to the FS-iA10B

This is the constraint that shapes the whole design and is the single most important thing for a reviewer to understand.

- The FS-iA10B receives **FlySky AFHDS 2A**: a proprietary, frequency-hopping GFSK protocol in 2.4055–2.475 GHz, implemented on an Amiccom A7105-class radio.
- The DX-LR41-24T12S is a **Semtech SX1281** LoRa/FLRC transceiver behind a UART.

- These are not interoperable at any layer. A DX-LR41 cannot transmit a valid AFHDS 2A packet, so an FS-iA10B cannot be controlled by one.

There were three ways forward (recorded here for the review, since this is the decision the ECR hinges on):

Option	How it works	Verdict
A. Keep the FS-iA10B, add a 4-in-1 multiprotocol module driven by the pad node	Well-trodden (open-source AFHDS 2A implementations exist for the A7105). Keeps the receiver and its hardware failsafe.	Rejected: two RF hops, +$40–70 for the module, the AFHDS 2A timing is not something to be maintaining on a liquid rocket, and the transmitter ID/pairing must be cloned.
B. Drop the FS-iA10B, pad node drives the servos directly (chosen)	One RF hop, ~$10 of extra electronics, full control of failsafes, timing and logging.	**Chosen.** Costs: the receiver's certified failsafe behaviour must be re-implemented and proven (see §6 and the test plan).
C. Keep the FS-i6X as a PPM→AFHDS "modem" and inject PPM into its trainer port	Uses only hardware already owned; the i6X does the AFHDS transmitting.	Rejected as the primary design: the i6X becomes a single point of failure inside the box, its trainer-mode channel counts are undocumented for 10 channels, and it keeps the closed protocol in the loop. Noted here because it is a cheap bench way to validate Option B against a known-good receiver.

Option B was selected by the design owner. This ECR documents its consequences honestly rather than hiding them.

4.3 The radio module

DX-LR41-24T12S — Shenzhen DX-Smart, Semtech SX1281, 2.4 GHz.

Parameter	Value	Source
Supply	3.3 V (module); break-out boards usually accept 5 V and regulate	vendor
UART	default 9600 8N1; 2400…115200 selectable (`AT+BAUD 0..6`)	串口应用指导 v2.0
Air rate presets	`AT+LEVEL 0..7` LoRa (0.5 → 150 kbit/s), `AT+LEVEL 8` FLRC (1.04 Mbit/s)	ibid.
Default preset	LEVEL 3 = SF8 / BW 406 kHz / 10 kbit/s, ≈2.2 km quoted LOS	ibid.
Channels	`AT+CHANNEL 00..32` = 2400 MHz + 2 MHz × n; default `0C` = 2424 MHz	ibid.
Transmit modes	`AT+MODE 0` transparent, `1` fixed-point (dst addr + dst channel + data), `2` broadcast	ibid.
Addressing / "encryption"	`AT+MAC` (2 bytes, used by fixed-point), `AT+OPENKEY`, `AT+KEY 0..65535`	ibid.
Chunking	`AT+PACKET 0..3` = 32/64/128/230 bytes	ibid.
Power	`AT+POWE 0..13` dBm, default 13 dBm (~20 mW)	ibid.
Handshake pin	`AUX` high while busy, asserted 2–3 ms before received data	ibid.
FLRC caveat	`LEVEL 8`: payload ≥ 6 bytes, ≤ 127 bytes, key feature unavailable	ibid.

Provisioning used by the firmware (both nodes write these at boot and verify with `AT+HELP`):

```
AT+MODE1            fixed-point transfer
AT+LEVEL3           10 kbit/s LoRa, ~2.2 km LOS  (configurable)
AT+CHANNEL0C        pad listens on 2424 MHz
AT+OPENKEY1 / AT+KEY51042
AT+MAC00,02         pad node address; TX node is 00,01
AT+PACKET1          64-byte chunks (our longest frame is 34 bytes on air)
AT+POWE13           13 dBm
AT+SLEEP2           high-timeliness mode: always listening
AT+RESET            commit
```

Two-channel scheme. The module is half-duplex. To keep the 20 Hz command stream and the 10 Hz telemetry from colliding, each module listens on one channel and transmits on the other using fixed-point addressing: pad listens 2424 MHz (`0C`) / transmits 2428 MHz (`0E`); TX node the reverse. This must be confirmed on the bench (test T-07); if a firmware revision ignores the destination channel the fallback is `-DGSE_SINGLE_CHANNEL` (both ends on `0C`) with telemetry at 5 Hz and the 20 Hz command stream reserved.

Airtime budget. 22 bytes on air per command frame (19-byte frame + 3-byte fixed-point prefix):

Preset	Modulation	Airtime per frame	20 Hz stream uses	Quoted LOS range
3 (default)	LoRa SF8 / BW406	≈ 27 ms	54 % — works, but leaves little margin	2.2 km
5	LoRa SF6 / BW806	≈ 7 ms	14 %	~1 km (extrapolated)
7	LoRa SF5 / BW1625	≈ 3 ms	6 %	~0.5 km
8	FLRC 1.04 Mbit/s	≈ 0.3 ms	0.6 %	~0.3–0.5 km

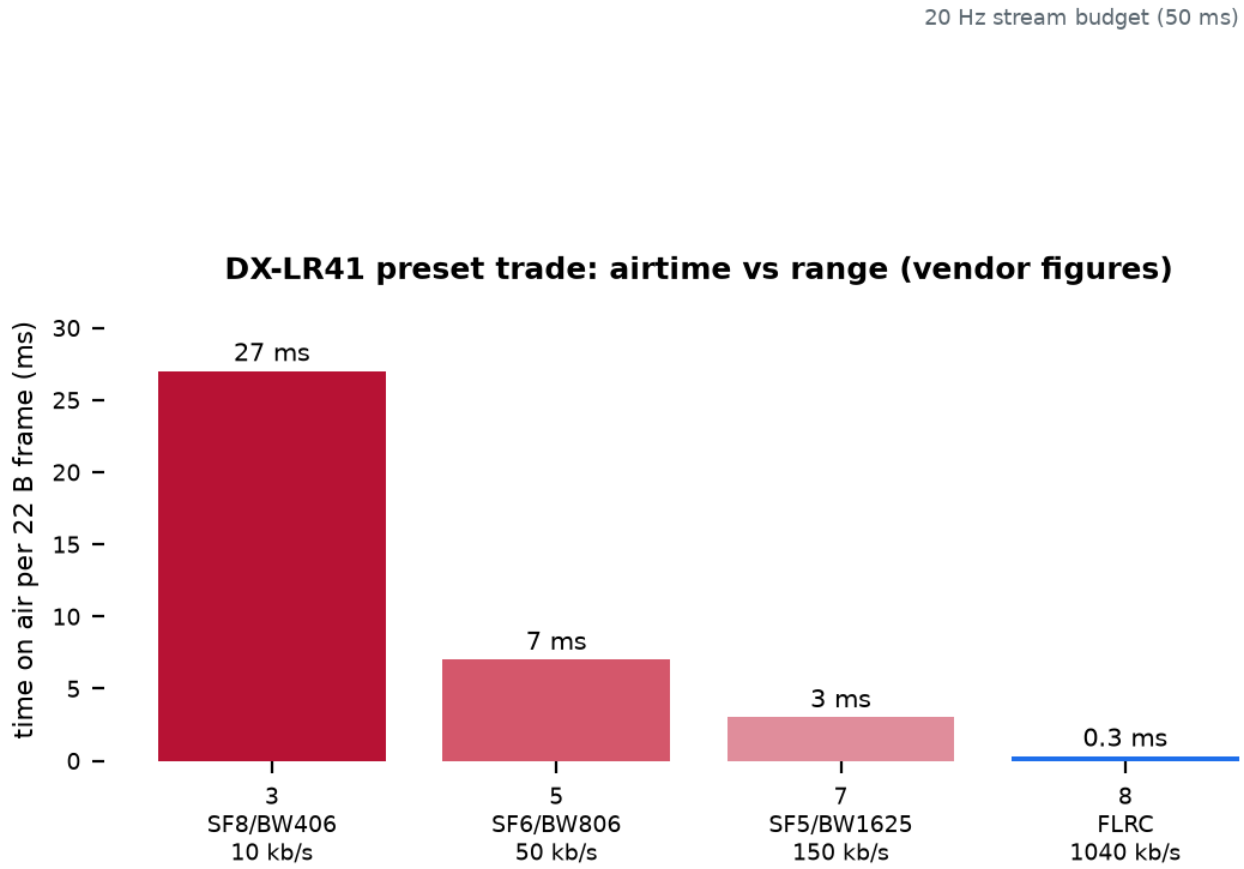


Figure 5 — Airtime trade. Recommended starting point is LEVEL 3 at 20 Hz, dropping to LEVEL 5 after the range test if the site layout is short.

Range quoted by the vendor is line-of-sight with an unobstructed Fresnel zone. The handbook's operating distances are 100 ft (30 m) standoff, or a bunker a few hundred metres away at FAR/RRS; the stock AFHDS 2A link is quoted at 500–1500 m. The LoRa link therefore has comparable or better margin at 13 dBm, but this must be measured, not assumed — see test T-05.

4.4 Wiring and power

Figure 6.1 (revised): GSE electrical schematic

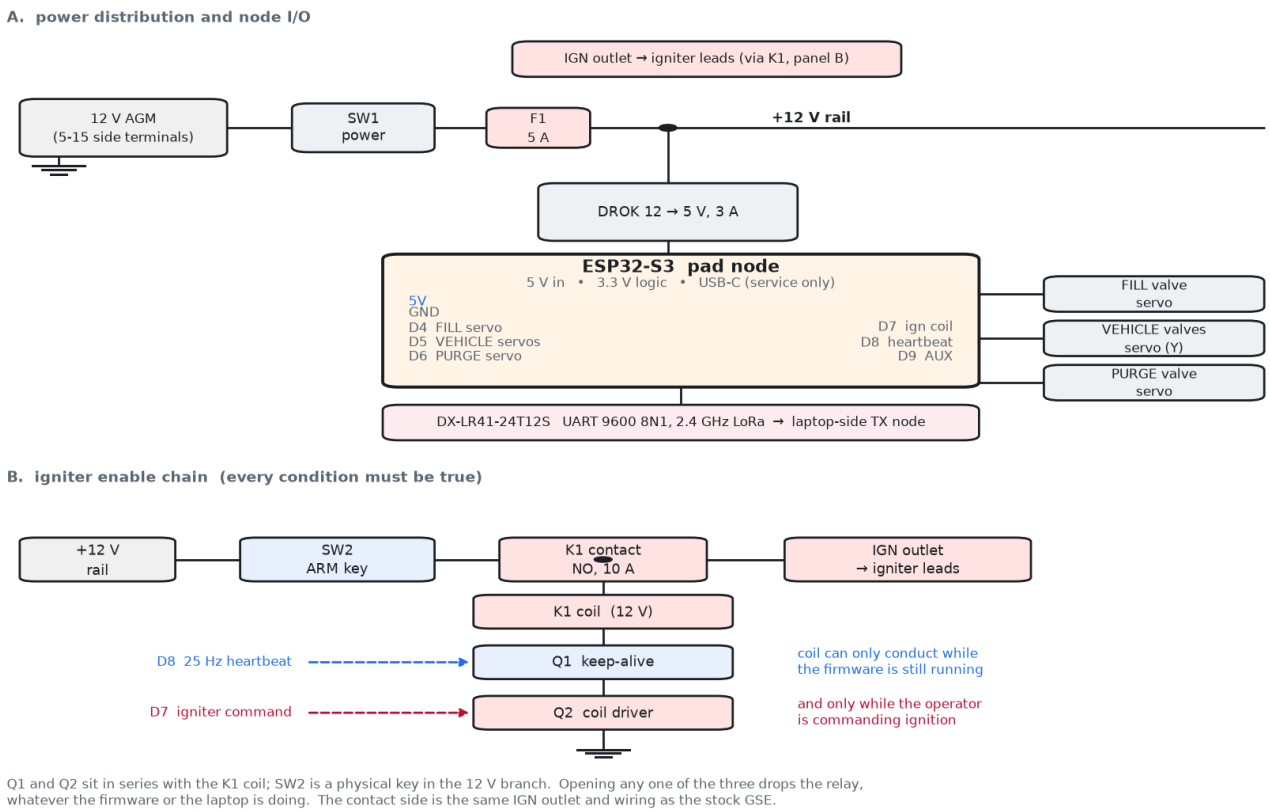


Figure 2 — Revised Figure 6.1. Panel A: power distribution and node I/O. Panel B: igniter enable chain.

Pad node pinout (ESP32-S3 DevKitC-1 / N16R8 class board):

Signal	GPIO	Notes
FILL servo PWM	D4 (GPIO 4)	50 Hz, 1000–2000 µs, 12-bit LEDC
VEHICLE servos PWM (Y-harness)	D5 (GPIO 5)	same, both servos share one signal as in the stock GSE
PURGE servo PWM	D6 (GPIO 6)	same
Igniter relay	D7 (GPIO 7)	drives the K1 coil-driver MOSFET gate
Keep-alive heartbeat	D8 (GPIO 8)	25 Hz square wave → Q1
DX-LR41 UART RX / TX	D16 / D17 (UART1)	9600 8N1, crossed to the module's TXD/RXD
DX-LR41 AUX	D18	input, pull-up
Status LED	D48	optional WS2812

TX node — two acceptable implementations. The laptop-side node is deliberately the dumb end of the link: it paces, filters and forwards, and holds no safety state that matters. It can therefore be built from either board without touching the pad, the protocol, the GUI or any procedure beyond the cable that lands on the laptop.

	ESP32-S3 (as specified)	NUCLEO-H753ZI (MB1364E) option
Laptop port	native USB-CDC	ST-LINK VCP, plain COM port, no driver install

	ESP32-S3 (as specified)	NUCLEO-H753ZI (MB1364E) option
Radio port	UART1 D16/D17, AUX D18	USART2 PD5 (CN11-41 Tx) / PD6 (CN11-43 Rx), AUX PD7 (CN11-45)
Supply	USB-C 5 V	USB-C 5 V (ST-LINK) or E5V/VIN
Debug	USB serial	SWD — breakpoints on a link you are range-testing
Firmware	`env tx-node`, 335 KB flash	`env tx-nucleo`, 33.9 KB flash, or CubeIDE HAL port
If it dies	link lost, pad goes safe	identical

Both builds execute the same portable core (`firmware/src/common/gse_tx_core.c`) and the same 20 Hz pacing, 200 ms stale-frame rejection and 300 ms laptop dead-man, so the pad cannot tell them apart. Details in `firmware/stm32/README.md`; the provisioning commands, the four wires and the `--port` argument are the only things that change.

Two traps worth printing on the sheet: the Nucleo's VCP is **USART3 (PD8/PD9)** through solder bridges SB5/SB6, not USART2, and its Arduino-header **D0/D1 are PG9/PG14 and are not a UART**. The radio lives on USART2 at PD5/PD6, which appears on the morpho header as CN11 pins 41/43.

Power. 3.3 V logic. The DX-LR41 must be fed 3.3 V (do not feed it 5 V unless the carrier board has its own regulator — check the silkscreen). Servos keep their existing 5 V supply from the DROK regulator. Total added load: ≈ 90 mA (module TX peak ≈ 120 mA) + two ESP32-S3 at ≈ 60 mA. The existing 3 A buck is adequate; add 100 µF at each LR41 supply pin.

4.5 Protocol

Fixed 7-byte header, CRC-16/CCITT-FALSE, little-endian fields (full definition in `firmware/src/common/gse_protocol.h`, mirrored in `software/gse_control/protocol.py`):

```
+------+------+------+-----+-----+---------------+--------+
| 0x7E | ver  | type | seq | len | payload[len]  | crc16  |
+------+------+------+-----+-----+---------------+--------+
```

Frame	Direction	Rate	Size on air	Payload
`HELLO`	both	on connect	10 B	node id, firmware version
`CMD`	TX → pad	20 Hz	19 B	arm code, FILL/VEH/PRG/IGN requests, watchdog + igniter limits, sender time
`STATE`	pad → TX	10 Hz	31 B	actual output states, link age, counters, config in force
`CFG`	TX → pad	on change	15 B	servo travel per channel, watchdog, igniter limit, purge enable
`EVENT`	pad → TX	on edge	10 B	boot, link loss, arm, igniter on/off, config change

4.6 Failsafes

This is the part of the design that replaces the receiver. It is implemented in three independent layers:

Layer 1 — pad-node link watchdog (primary). Every accepted `CMD` stamps a local clock. Every 20 ms the pad recomputes its outputs from the last accepted command plus elapsed time. If the age of the last command exceeds the configured watchdog (default **1.0 s**, range 0.3–3.0 s), the outputs snap to:

```
FILL    = CLOSED
PURGE   = CLOSED
VEHICLE = OPEN      ← deliberately open, dumps propellant, exactly as the
IGNITER = OFF          handbook's FS-iA10B failsafe is configured
```

This is the same safe set that OP 8 ("Configure receiver failsafes") establishes on the iA10B, and it preserves the handbook's stated philosophy: loss of communication with the GSE automatically opens the vehicle propellant valves, returning the system to a safe state (§6.1).

Layer 2 — TX-node laptop watchdog. If the laptop stops sending frames for 300 ms (GUI crash, closed lid, unplugged cable), the TX node begins transmitting synthetic SAFE commands at 20 Hz on its own. The pad therefore sees a valid, fresh, safe command rather than silence, and layers 1 and 2 both drive the same end state. Stale laptop frames (more than 200 ms after the previous frame) are never relayed.

Layer 3 — hardware (igniter only). The K1 coil is in series with **SW2 (physical arm key)** and two MOSFETs: **Q2** (commanded by D7) and **Q1** (driven by the 25 Hz keep-alive on D8 through an RC charge pump). Loss of the heartbeat → Q1 off → coil dead, regardless of firmware. The ESP32-S3 task watchdog (2 s) reboots a wedged loop into the safe boot state.

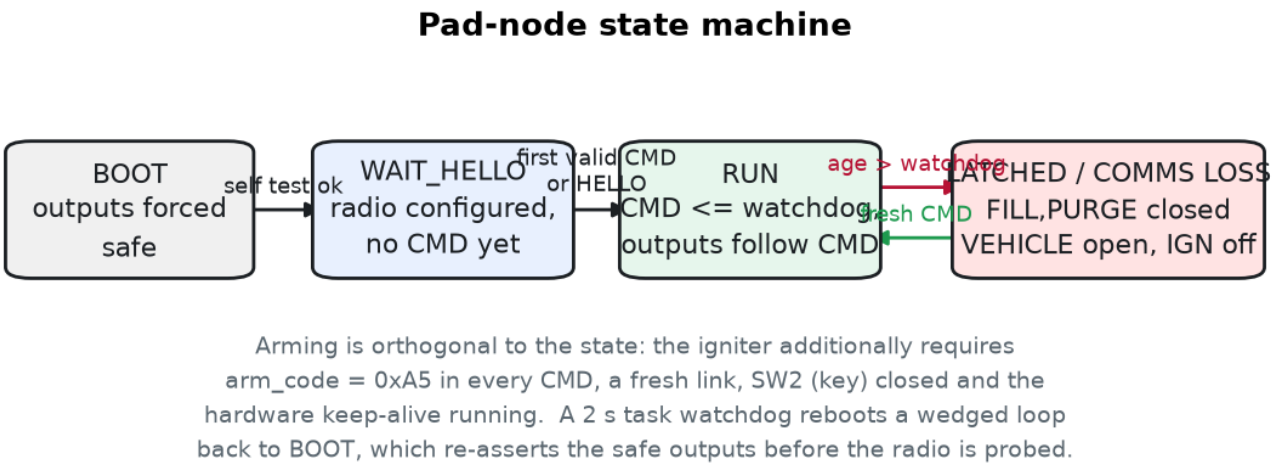


Figure 3 — Pad-node state machine. Arming is orthogonal to the state: it requires a code in every command, a fresh link, SW2 closed and the heartbeat running.

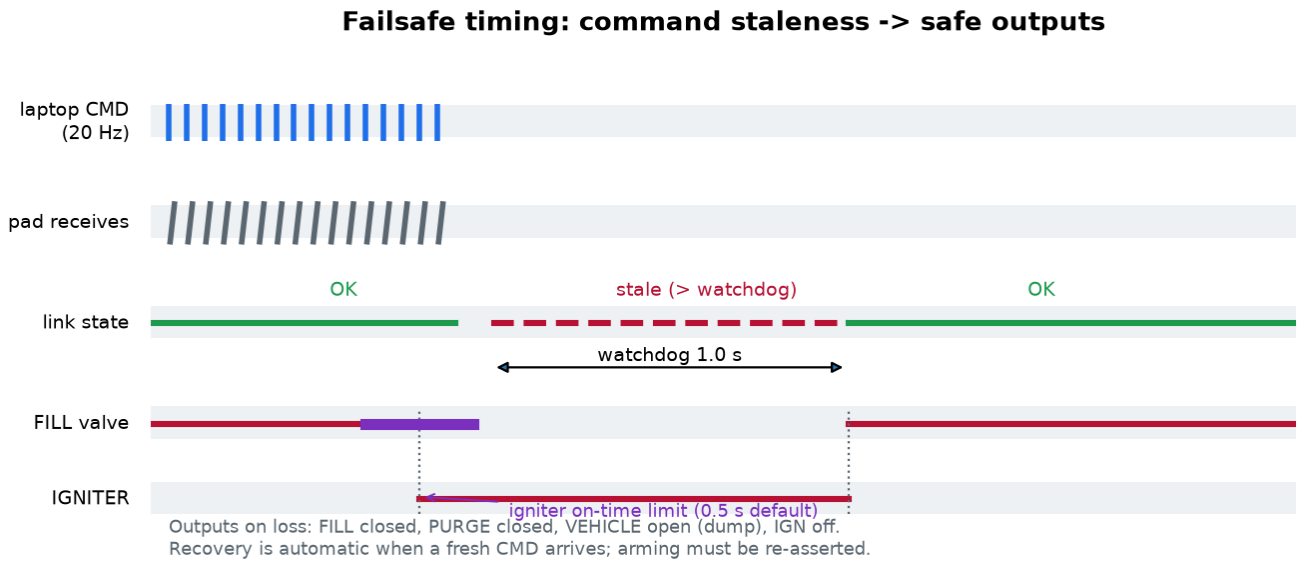


Figure 4 — Failsafe timing across a 1 s link blackout.

Igniter sequencing. One pulse per press: a fire command energises the relay for at most `ign_max_ds` (default **0.5 s**, range 0.1–2.0 s) and the pad then latches "spent" until the operator releases the control. A held button, a stuck command or a repeated frame cannot re-pulse the e-match.

Arming interlock. Identical in spirit to the handbook's SwC + stick lockout, but stricter:

1. the GUI's ARM control must be on,
2. the GUI must have a fresh pad state (it refuses to arm blind),
3. the `arm_code` (0xA5) must be present in every command frame — one frame without it drops the arm latch within 20 ms,
4. SW2 (physical key) must be closed,
5. the heartbeat must be running.

4.7 Configurable parameters (no re-flash required)

Sent from the GUI as a `CFG` frame and echoed back in every `STATE` frame:

Parameter	Default	Range	Effect
FILL travel	45°	5–90°	servo throw each side of neutral (= valve 0/90°)
VEHICLE travel	45°	5–90°	ditto; fine-tunes both valves of the Y-harness together
PURGE travel	45°	5–90°	ditto
Link watchdog	1.0 s	0.3–3.0 s	layer-1 timeout
Igniter limit	0.5 s	0.1–2.0 s	hard relay on-time
Purge enabled	on	on/off	when off the purge channel is forced closed in firmware

5. Bill-of-materials change

5.1 Removed

Line	Part	Description	Was	Context
1	`DTUS_FSI6X_10CH`	FlySky FS-i6X 10CH transmitter w/ FS-iA10B receiver	$59.99	Controller
—	—	FS-iA10B receiver (part of line 1)	—	Receiver in box
5	`LCATMC1569`	PWM-controlled relay module	$11.99	Ignition switch — **replaced** by an on-board relay (see 5.2)
9	`7526K23`	Nylon wall plate, "IGN"/"SRV" labels	$0.73	Superseded by GUI labelling; keep as service outlet cover
10	`YXQ`	3-pin servo extension cables, 1 m ×10	$0.00	Servos now land on the pad node headers; keep 2 for the Y-harness

5.2 Added

Indicative prices, USD, single quantity, Nov 2026. Vendors are examples, not endorsements.

#	Part	Description	Qty	Unit	Cost	Purpose
A1	ESP32-S3-DevKitC-1 (or N16R8 "SuperMini")	ESP32-S3 devkit with native USB, 8 MB flash	2	$8.00	$16.00	TX node + pad node [1]
A2	DX-LR41-24T12S	2.4 GHz SX1281 UART module, 13 dBm, IPEX/SMA	2	$9.50	$19.00	Radio pair
A3	2.4 GHz antenna, SMA, 3 dBi	one per module (module ships with IPEX pigtail option)	2	$3.00	$6.00	Range
A4	Proto board 5×7 cm + headers	carrier for the pad node (servo headers, screw terminal for 12 V)	1	$3.00	$3.00	Robust wiring
A5	Relay module, 1-channel, 12 V coil, 10 A, opto-isolated, **with flyback diode**	replaces `LCATMC1569`; driven from D7	1	$6.00	$6.00	Igniter switching
A6	IRLZ44N or AO3400A logic-level MOSFET ×3	Q1, Q2 + spare	1	$1.50	$1.50	Keep-alive + coil driver
A7	1N5819 / 1N4007	flyback across K1 coil	2	$0.20	$0.40	Coil transient
A8	Key switch (2-position, SPST, keyed)	SW2 arm key	1	$6.00	$6.00	Physical arm interlock
A9	5 A blade fuse + inline holder	F1	1	$3.00	$3.00	Over-current
A10	100 µF electrolytic + 100 nF ceramic	module supply decoupling	4	$0.30	$1.20	RF supply stability
A11	USB-C panel-mount extension, 1 m	laptop cable lands on the toolbox face	1	$7.00	$7.00	Robust daily use
A12	Heat-shrink, hookup wire 22 AWG, servo crimp kit	consumables	1	$6.00	$6.00	Wiring
A13	USB power bank (5 V, 2 A) optional	powers the TX node if the laptop port is unreliable	1	$12.00	$12.00	Redundancy

[1] Either devkit may be a **NUCLEO-H753ZI** (MB1364E), which is already on the bench for avionics work. The price column is unaffected for budgeting; use $0 for the Nucleo if it is already owned. See §4.4 for the pinout.

The TX node may be a NUCLEO-H753ZI in place of one ESP32-S3 (line A1) if the builder already owns one; the BOM delta then falls by $8.00. Nothing else in the BOM changes — the same DX-LR41, antenna, decoupling and cable are used.

Delta: removed $71.98 (lines 1 and 5) and added $86.50 (lines 38–50) → **+$14.52 net per GSE**, $529.45 → $543.97, excluding the laptop (the handbook already assumes one for data reduction). These are the same figures as the §6.2 table in `docs/INSERT_SECTION_VI.pdf`, which is the number of record.

5.3 Retained

12 V AGM battery, DROK 12→5 V regulator, GSW-17 power switch, NEMA outlet pair (kept as a labelled service outlet, or removed for a cleaner box), eye bolts, toolbox, all OP 1–OP 7 hardware, all fluid-system parts.

---

6. Safety analysis of the change

6.1 Hazards introduced, and their controls

#	Hazard	Cause	Control	Residual risk
H1	Valve held open after link loss	radio dropout, dead laptop	pad watchdog 1.0 s (layer 1) + TX-node 300 ms safe stream (layer 2)	Low — two independent timers
H2	Igniter energised by a hung or reused command	firmware defect, stale frame	arm code in every frame, one-shot latch, 0.5 s hard limit, Q1 heartbeat, SW2 key	Low
H3	Igniter fires on boot / reset	uncontrolled GPIO at boot	boot state forces outputs safe before the radio is configured; relay drives low in the first instructions of `setup()`; SW2 open during boot	Very low
H4	Servo PWM glitch on reset	undefined GPIO	LEDC attached and outputs written before `Serial.begin()`; DS3225-class servos hold position with no signal	Very low
H5	Lost radio link not noticed by the operator	silent dropout	pad reports link state at 10 Hz; GUI banner turns red after 1.5 s of silence; TX node logs stale counts	Low
H6	RF interference from another 2.4 GHz user	site congestion	LoRa spreading gain, KEY/address filtering, two-channel scheme, RSSI telemetry not yet enabled	Medium — measure at the site (T-05)
H7	Wrong servo endpoints stall a valve servo	user-entered travel > mechanical stop	GUI clamp 5–90°, firmware clamp, travel applies symmetrically	Low
H8	Purge valve opened with the vehicle connected	operator error	`purge_enabled` must be ticked in the GUI each session; firmware forces the channel closed otherwise	Low
H9	Laptop battery dies mid-fill	operator error	layer 1 and layer 2 both safe the pad; the GUI logs every frame to CSV	Low

6.2 Comparison with the stock receiver's failsafe

Behaviour	FS-iA10B (stock)	New pad node
Trigger	loss of AFHDS 2A packets	no valid `CMD` for > watchdog (0.3–3.0 s, configurable)
Failsafe state	per-channel, set in the transmitter menu	fixed and tested: FILL/PURGE closed, VEHICLE open, IGN off

Behaviour	FS-iA10B (stock)	New pad node
Verification	turn the transmitter off and watch (OP 8 step 7–8)	GUI read-back + test T-04 + the pad's own state telemetry
Independent of the receiver MCU	no — it is the receiver MCU	partly: the igniter also drops on loss of the heartbeat (hardware)
Configurable in the field	yes, in the transmitter menu	yes, in the GUI
Logged	no	yes, CSV, every frame

6.3 Regulatory

2.4 GHz operation at 13 dBm (20 mW) EIRP is within FCC Part 15.247 limits for the US (1 W conducted) and ETSI EN 300 328 for the EU. **Duty cycle:** at LEVEL 3 the 20 Hz command stream occupies ≈54 % of the time; the FCC allows this, but ETSI EN 300 328 requires either a duty-cycle limit (typically 10 %) or Listen-Before-Talk / DAA for certain bands and powers. Set `AT+LBT=1` (with `AT+LRSSI` tuned to the measured noise floor, `AT+ERSSI`) for EU operation, or use LEVEL 5/7 to stay under 10 %. The module exposes both knobs. This must be checked by the operating team for their jurisdiction; it is a **new** regulatory consideration that did not exist with the AFHDS 2A radio.

---

7. Verification

The unit and protocol tests are already implemented in this package and pass:

```
tests/test_protocol.c          6 tests   (CRC vector, framing, corruption/resync,
                                           failsafe matrix, 500-frame noisy stream)
tests/host_hal_stub.c         29 tests   (the portable TX core compiled unmodified on the
                                           host: provisioning, forwarding, stale reject,
                                           dead-man SAFE stream, text console, uplink)
software/tests/test_protocol.py 9 tests  (byte-identical golden frames vs firmware,
                                           parser parity, failsafe parity)
software/tests/test_sim_loop.py 4 tests  (live link, watchdog safe-out, igniter
                                           interlock + time limit, purge gating)
```

Firmware builds clean for all three targets (`pio run -e tx-node`, `-e pad-node`, `-e tx-nucleo`; ESP32 RAM ≈ 7 %, flash ≈ 10 %; Nucleo 2.5 KB RAM, 33.9 KB flash). The STM32Cube HAL port is additionally compiled header-only with `sh firmware/stm32/syntax_check.sh`, so it is checked even when no CubeMX project exists on the machine.

The bench and range test plan — including the acceptance criteria that must be met before the first live nitrous fill — is in **`docs/TEST_PLAN.md`**. The minimum gate to fly is T-01…T-06 (bench) plus T-08/T-09 (range) signed off.

---

8. Implementation

8.1 Work breakdown

Phase	Task	Est. effort
1	Order parts; flash both nodes; bench power-up	1 evening

Phase	Task	Est. effort
2	Bench bring-up: packet test, failsafe tests T-01…T-04	1 evening
3	Assemble pad-node carrier board, mount in the toolbox, wire servos/relay	1 weekend
4	Bench integration with the real servos and a dummy load for the igniter (T-06)	1 evening
5	Range test with the GSE in its flight configuration (T-05, T-07)	½ day at the site
6	Full dress rehearsal with a spare servo and a lamp on the IGN outlet (T-08), then the handbook's own §7 checkouts	½ day
7	Update the handbook pages and the OP 8 procedure	1 evening

8.2 Rollback plan

Because the servo harness and the igniter lead are unchanged, rollback is:

1. re-fit the FS-iA10B and plug the three servo leads and the PWM relay back in (10 minutes),
2. re-run OP 8 (`Configure receiver failsafes`) on the stock transmitter,
3. the pad-node carrier stays in the box, unplugged.

The FS-i6X and FS-iA10B are **retained as spares** even though they are removed from the active design. They are the qualification reference for the new link (see T-02).

8.3 Training

The operator-facing difference is a GUI instead of a transmitter. Two things must be trained explicitly:

- the **ARM** action is now a GUI control and a key switch, not a switch under a cover;
- the **igniter** is hold-to-fire with a hard limit, and releasing the control (or pressing `Esc`) drops it immediately.

A one-page quick-reference card is included as `docs/OPERATOR_CARD.md` .

---

9. Risks and open items

#	Risk / open item	Impact	Mitigation / next step	Owner
R1	Fixed-point destination-channel behaviour not yet verified on the bench	telemetry collisions with the 20 Hz command stream	T-07; fallback `-DGSE_SINGLE_CHANNEL` at 5 Hz telemetry	GSE lead
R2	LoRa airtime at LEVEL 3 leaves 54 % channel occupancy	latency growth if telemetry is enabled at 10 Hz	measure; drop to LEVEL 5 on short sites	GSE lead
R3	ESP32-S3 is a 3.3 V devkit on a breadboard-adjacent carrier	vibration, connector creep	solder the carrier, strain-relieve the servo headers, hot-glue the USB-C extension	build team
R4	Arduino-ESP32 core 3.x LEDC API differs from 2.x	build fragility	firmware supports both APIs (`ESP_ARDUINO_VERSION_MAJOR`)	firmware
R5	Igniter relay module needs a real flyback and a snubber	contact welding, EMI resetting the MCU	A5/A7; add a 100 nF across the contacts; test with a lamp before an e-match	build team
R6	Operator unfamiliarity with a computerised control	procedural error under time pressure	rehearsals T-08/T-09, quick-reference card, keep the physical SW2 key	launch ops

#	Risk / open item	Impact	Mitigation / next step	Owner
R7	No independent telemetry of actual valve position	a stripped servo spline would be invisible	audible/visual check in the on-pad checkouts; optional micro-switch feedback in a future revision	future work
R8	The DX-LR41 is not a certified module in every jurisdiction	legal	jurisdiction check (§6.3); the vendor offers a CE/FCC variant of the SX1281 family	launch ops

10. Document changes

10.1 Section VI — text replacements

§6.1, paragraph 2 (control system description) — replace the sentence beginning "The control system is taken straight from a much more well-developed and commercially available field – hobby R/C aircraft…" with:

> The control system is a purpose-built, laptop-driven link. A small Python application on the operator's laptop sends command frames over USB-C to an ESP32-S3 radio node inside the GSE box, which relays them over a 2.4 GHz LoRa link (Semtech SX1281, DX-LR41-24T12S modules) to a second ESP32-S3 at the launch pad. The pad node drives the three valve servos and the igniter relay directly and reports the actual state of every output back to the laptop at 10 Hz. Channel numbering is preserved from the original design — FILL on Ch5, VEHICLE on Ch6, PURGE on Ch8, IGNITER on Ch3 — but is software-defined and can be re-mapped without re-wiring. Because the link is not a certified hobby receiver, all failsafe behaviour is implemented in the pad node and verified per §7 below: loss of communication for longer than the configured watchdog (1.0 s default) closes the fill and purge valves, opens the vehicle valves, and de-energises the igniter. This mirrors the safe state the stock receiver was configured for.

§6.1, paragraph 3 (control system composition) — replace "The control system consists of a transmitter, receiver, battery, voltage regulator, and electrical relay." with:

> The control system consists of the operator's laptop, two ESP32-S3 radio nodes, two DX-LR41-24T12S modules, the battery, the voltage regulator, the arm key switch, and the igniter relay.

Figure 6.1 — replace with Figure 2 of this ECR.

§6.2 table — apply §5 above; renumber line items 1–37 accordingly and add the new subtotal. The insert already carries the finished table (pages 240–241).

OP 6 steps 1, 2, 7–15 — replaced by the new procedure in §10.2.

OP 8 — replaced in full by §10.2 (Bring-up) and §10.3 (Configuration).

§7.2.2 steps 18 and **§7.3.2 steps 1–3, 7–11** — see §10.4.

§7.5.2 step 4a (caution about not cycling the transmitter) — the equivalent caution for the new system is:

> Do not close and re-open the laptop GUI during this time. Restarting the application re-runs its link handshake; although the pad stays safe while the link is down, an accidental re-arm must be avoided. If three attempts to command the vehicle valves open are unsuccessful, close the application and leave it closed until the vehicle and pad are completely safed.

10.2 New §6.3 OP 6A — Assemble the control system (replaces OP 8)

Gather the following:

QTY	Part number	Description
1	ESP32-S3-DEVKITC-1	ESP32-S3 devkit (TX node)
1	ESP32-S3-DEVKITC-1	ESP32-S3 devkit (pad node)
2	DX-LR41-24T12S	2.4 GHz LoRa module
2	ANT-24G-SMA-3DBI	2.4 GHz antenna
1	RLY-12V-1CH-OPTO	12 V, 10 A relay module with flyback
1	SW-KEY-SPST	Key switch, 2-position
3	IRLZ44N	Logic-level MOSFET (Q1, Q2, spare)
2	1N5819	Flyback diode
1	FUSE-5A-INLINE	5 A fuse and holder
1	USB-C-PANEL-1M	Panel-mount USB-C extension
—	—	22 AWG hookup wire, crimp terminals, heat-shrink
1	4112T22	existing fill valve assembly (unchanged)

Procedure

1. Flash the two nodes from a computer with PlatformIO installed: `cd firmware pio run -e tx-node -t upload # first board pio run -e pad-node -t upload # second board, one at a time` Label the boards **TX** and **PAD** as soon as they are flashed. Label before the radio modules are fitted.
2. Fit each DX-LR41 to its board: 3V3 → module VCC, GND → GND, D16 → module TXD, D17 → module RXD, D18 → module AUX. **Do not** connect the module to 5 V.
3. Fit the antenna to each module. Never power a module without an antenna.
4. On the pad-node carrier board, wire, in this order: a. servo headers SIGNAL → D4/D5/D6, +5 V → existing servo rail, GND → common; b. relay module IN → D7, VCC → 12 V (through F1), GND → common; c. the SW2 key switch in series with the relay's 12 V supply, **upstream** of the relay contact, so an open key removes coil power; d. the keep-alive components (Q1, RC, D8) per Figure 2, panel B; e. 100 µF + 100 nF at the module supply.
5. Mount the carrier inside the toolbox on the same stand-offs the FS-iA10B used. Route the antennas out through the existing antenna cover (`CVR-ANT-2X8C` is still usable — the two SMA bulkheads replace the two wire antennas) and keep them at least 30 mm apart and away from the servo wiring.
6. Install the USB-C panel-mount extension through the toolbox face; the laptop cable lands there. This replaces the plug-in transmitter.
7. Re-use the existing 12 V battery, DROK regulator, GSW-17 power switch and outlet pair. The `SRV` outlet becomes a service outlet; label it accordingly.
8. **Before** connecting the servos, perform the bring-up checks in OP 6B.

10.3 New §6.3 OP 6B — Node bring-up and configuration (replaces OP 8)

1. Power the TX node from the laptop and start the application: `python software/run_gui.py` (or `--port /dev/ttyACM0` / `--port COM7`).
2. Select the TX node's port (it appears as an ESP32-S3 CDC device) and press **Connect**. The event log must show `connected` and then `HELLO_OK` once the pad node answers.
3. Power the pad node (SW1 on). Verify: a. within 1 s the banner changes from `NO PAD TELEMETRY` to `LINK OK – NOT ARMED`; b. `last valid CMD` reads < 100 ms and `rejected frames` stays at 0.
4. **Failsafe check (must pass before anything is connected to the IGN outlet):** a. with no servo and no igniter connected, note the banner; switch off SW1 and watch the GUI banner turn red within 1.5 s; b. switch SW1 back on; the pad must return to the safe state reported as `VEHICLE OPEN, FILL CLOSED, PURGE CLOSED, IGN off`; c. unplug the laptop cable: the pad must declare `COMMS LOSS – PAD SAFE` within 1.5 s.
5. **Servo endpoint adjustment** (do this before final assembly of each ball valve, exactly as the original OP 8 required): a. connect only the FILL servo to D4; b. tick `purge valve fitted` only if a purge valve exists; c. operate FILL and watch the valve handle: it must be perpendicular to the body when `CLOSED` and parallel when `OPEN`; d. if it cannot reach either position at 45° travel, change FILL travel in the GUI (5–90°) and press **Apply to pad**; if the horn still cannot reach the stop, remove the horn and re-fit one spline over; e. repeat for VEHICLE (D5, both valves through the Y-harness) and PURGE (D6).
6. **Igniter check** (dummy load, never an e-match): a. fit a 12 V lamp or a 100 Ω/5 W resistor across the IGN outlet; b. with ARM off, press-and-hold the fire control: the lamp must stay dark and the log must record `igniter pressed but ARM is off`; c. switch ARM on; the banner must turn green and read `ARMED`; d. press-and-hold: the lamp must light; the state read-back must show `IGNITER ENERGISED`; after the configured limit (0.5 s) the lamp must go out even with the button held; e. release, press again: it must **not** re-fire until the control is released (one pulse per press); f. press `Esc`: ARM drops, the lamp goes dark.
7. **Dress the configuration** for the flight: set the watchdog (1.0 s recommended for the pad, 0.5 s for bench work), the igniter limit (0.5 s), and the servo travels verified above. Press **Apply to pad** and confirm the pad echoes `CFG_CHANGED` in the event log.
8. Disconnect, power everything down, and record the configuration in the launch log for the day.

10.4 Section VII changes

- **§7.2.1(e)** "Transmitter for GSE control" → "Operator laptop with the GSE application installed, charged, and the TX-node cable".
- **§7.2.2 step 18 (optional checkout)** → start the application, connect, and confirm `LINK OK – NOT ARMED` before leaving the pad.
- **§7.3.2 steps 1–3** → replace the transmitter boot sequence with: "1. Start the GSE application on the laptop. 2. Connect to the TX node. 3. Switch on SW1 (pad). Verify the banner reads *`LINK OK – NOT ARMED`* and the *`last valid CMD`* age is < 100 ms."
- **§7.3.2 steps 7–11** → the FILL/SRV/IGN checks are unchanged in intent; operate them from the GUI and confirm the read-back panel matches what is physically observed. The spark check becomes: with ARM off, confirm no spark on the leads; ARM on, press and hold IGN, confirm spark within 0.1 s; release, confirm no spark.
- **§7.3.3 step 3** — "Confirm that the Operator has physical possession of the GSE control transmitter" → "Confirm that the Operator has physical possession of the laptop and that ARM is off with all switch positions reported CLOSED/SAFE".
- Add to **§7.3.3**: "Close the SW2 arm key and keep the key with the Operator. SW2 must be opened again before any person approaches the pad."

11. References

1. Half Cat Rocketry, Mojave Sphinx — Build, Integration, and Launch Guidebook, HCR-5100 rev 01, 17 Aug 2024 — Section VI, OP 6 and OP 8, §7.2–7.5.
2. Shenzhen DX-Smart, DX-LR41-24T12S 串口应用指导 (Serial application guide), v2.0, 2025-08-11 — AT command set, LEVEL table, channel table, AUX behaviour.
3. Semtech, SX1280/SX1281 datasheet — 2.4 GHz transceiver, LoRa/FLRC/GFSK, ranging.
4. FlySky, FS-i6X / FS-iA10B user manual — AFHDS 2A, RX failsafe configuration, PPM output.
5. FCC 47 CFR Part 15.247; ETSI EN 300 328 — 2.4 GHz wideband transmission limits and duty-cycle/LBT requirements.
6. Espressif, ESP32-S3 technical reference — LEDC PWM, task watchdog, native USB-CDC.

Appendix A — File map of this change package

```
ECR-01_GSE_radio_replacement.md      this document
ECR-01_GSE_radio_replacement.pdf     same, paginated
docs/INSERT_SECTION_VI.pdf           insert-ready replacement pages 236-255, formatted to
                                     mirror the HCR-5100 layout (source: .html)
docs/TEST_PLAN.md                    bench + range test plan with acceptance gates
docs/OPERATOR_CARD.md                one-page range quick-reference
docs/fig_*.png                       figures 1–5 + GUI screenshot
firmware/                            PlatformIO project, three environments
  src/common/gse_protocol.h          wire protocol + failsafe rules (single source of truth)
  src/config.h                       all builder-editable constants
  src/dxlr41.h                       AT-command driver with verification
  src/pad/pad_node.cpp               pad node: safety logic, PWM, igniter, telemetry
  src/common/gse_hal.h               the seven platform services a TX node must supply
  src/common/gse_tx_core.c/.h        portable TX bridge: pacing, stale reject, safe stream, console
  src/tx/tx_platform_esp32.cpp       TX node platform layer (Arduino / ESP32-S3)
  src/stm32/tx_platform_nucleo.cpp   TX node platform layer for env tx-nucleo (Arduino / STM32H7)
  stm32/tx_platform_stm32.c          STM32Cube HAL port for CubeIDE projects
  stm32/README.md                    Nucleo pin map, build options, bench checks, rollback
  stm32/syntax_check.sh              header-only compile check of the HAL port
software/
  run_gui.py                         launcher
  gse_control/protocol.py            host mirror of the protocol (+ golden vectors)
  gse_control/link.py                serial transport + simulator + CSV logger
  gse_control/sim.py                 pad-node model used for rehearsal and CI
  gse_control/controller.py          all GUI logic, headless-testable
  gse_control/app.py                 tkinter GUI
  tests/                             9 protocol tests + 4 closed-loop tests
tests/                               C unit tests for the protocol + host HAL stub for the core
tools/make_figures.py                regenerates every figure
hardware/wiring.md                   bench wiring sheet and connector pinouts
```